

DK LEARNING

GRADE 13

Computational Thinking Capstone

Level: **A/L Project** • Purpose: Learn before you practise

[← BACK TO LEARNING WEBSITE](#)

In this guide

1. Defining the Problem
2. Decomposition and Design
3. Implementation
4. Testing and Evaluation
5. Presenting and Ethics
6. Practice questions, answers and glossary



How to use this guide

Read each chapter slowly. Look at the example, then try the “Your turn” task **without copying**. Write your own notes on the lines. When you finish, answer the practice questions and check your work with the answer page. After that, open the Grade 13 activity on the website.

Learning goals

- Write clear requirements and success criteria
- Design the system with data flow and modules
- Implement and test the solution
- Evaluate the result and present it

Study plan

Step	What to do
1	Read the chapter and highlight key words
2	Study the worked example
3	Do the “Your turn” task on paper or on a computer
4	Write a short summary in your own words
5	Answer the practice questions, then check

Tip: Write your success criteria first. Then you will always know whether your solution is finished.

1 Defining the Problem

A capstone begins with a clear problem and requirements. Identify users, inputs, outputs, constraints and success criteria before coding. Ask: who is it for, what must it do and how will we know it works?

Separate functional requirements (what it does) from non-functional ones (speed, security, ease of use).

Key points

- Know the users
- List inputs and outputs
- Functional vs non-functional
- Define success criteria

Worked example

Problem: Teachers waste time taking attendance.

Users: teachers, admin

Success: take attendance for 40 students in under 2 minutes

Your turn

Write five functional requirements for a student-management system.

Check yourself

Explain each term in your own words:

Requirement

Constraint

My notes

2 Decomposition and Design

Design should explain data flow, modules, interfaces and important decisions. Decomposition breaks a big problem into smaller parts. Use diagrams such as data flow diagrams, flowcharts and wireframes.

Choose structures and algorithms that fit the problem rather than coding randomly.

Key points

- Break into modules
- Show data flow
- Plan the interface
- Record design decisions

Worked example

Modules: Input -> Validation -> Storage -> Processing -> Reporting

Your turn

Design modules for input, validation, storage, processing and reporting.

Check yourself

Explain each term in your own words:

Decomposition

Wireframe

My notes

3 Implementation

Build one module at a time and test it. Use meaningful names, comments and functions. Keep versions with version control. Handle errors and validate input.

Do not wait until the end to test.

Key points

- Small steps
- Readable code
- Version control
- Validate and handle errors

Worked example

```
def add_student(students, name):  
    name = name.strip()  
    if not name:  
        return False  
    students.append(name)  
    return True
```

Your turn

Implement and test one module from your design.

Check yourself

Explain each term in your own words:

Implementation

Refactor

My notes

4 Testing and Evaluation

Testing should include normal cases, boundary cases and invalid inputs. Record expected and actual results in a test table. Evaluation compares the finished solution with the original requirements and identifies future improvements.

Ask users for feedback.

Key points

- Normal, boundary, invalid
- Test table
- Compare with requirements
- Collect feedback

Worked example

Test	Input	Expected	Actual
1	"" (empty)	Rejected	Rejected
2	"Amal"	Accepted	Accepted

Your turn

Create a test table with six test cases for your capstone.

Check yourself

Explain each term in your own words:

Evaluation

Feedback

My notes

5 Presenting and Ethics

A good presentation explains the problem, solution, results and lessons learned. Show a short demo. Be honest about limits.

Think about ethics: protect user data, give credit for sources, and consider who might be harmed or left out.

Key points

- Problem, solution, results
- Demo clearly
- Be honest about limits
- Respect privacy and credit sources

Worked example

Presentation outline:

1. Problem
2. Design
3. Demo
4. Test results
5. Improvements

Your turn

Prepare a five-minute presentation outline for your project.

Check yourself

Explain each term in your own words:

Ethics

Stakeholder

My notes



Practice Questions

Part A - Choose the best answer

1. Capstone work starts with...
 - A) a clear problem
 - B) the colour theme
 - C) coding at once
2. Functional requirements describe...
 - A) what the system does
 - B) only colours
 - C) room size
3. Decomposition means...
 - A) breaking a problem into parts
 - B) deleting code
 - C) printing
4. Boundary tests check...
 - A) the edges of allowed values
 - B) random mouse use
 - C) colours
5. Evaluation compares results with...
 - A) the requirements
 - B) a menu
 - C) the weather
6. Ethics in computing includes...
 - A) protecting user data
 - B) ignoring users
 - C) copying all

Part B - Short answers

1. What are success criteria?

2. Name two parts of a test table.

3. Why decompose a problem?

4. Give one ethical issue in software.



Answer Page

Part A

1-A 2-A 3-A 4-A 5-A 6-A

Part B (sample answers)

1. Measurable statements showing when the solution works.
2. Input and expected result (also actual result).
3. Smaller parts are easier to design, build and test.
4. User privacy, fairness, credit for sources, accessibility.

Mini glossary

Requirement	A need the system must meet
Constraint	A limit such as time, cost or device
Decomposition	Breaking a problem into smaller parts
Wireframe	A simple sketch of a screen
Implementation	Turning a design into working code
Refactor	Improving code without changing what it does
Evaluation	Judging how well the solution meets its goals
Feedback	Opinions from users that help improve a solution
Ethics	Principles of right and wrong in technology
Stakeholder	Anyone affected by a system



Activity Preparation

Plan a small ICT solution, answer project questions and submit a structured coding / design challenge. Then open the Grade 13 activity.

Think before you start

- Explain one important idea from this grade in your own words.
- Give one real-life example where this topic is useful.
- What mistake could a beginner make, and how can it be prevented?
- Write the steps you will follow before starting the Grade 13 activity.

Ready checklist

- | | |
|---|--|
| ☐ I read every chapter | ☐ I tried the “Your turn” tasks |
| ☐ I understand the key terms | ☐ I checked my practice answers |
| ☐ I wrote my own notes | ☐ I am ready for the activity |

⚡ **START GRADE 13 ACTIVITY**

← **EXIT PDF / BACK TO LEARNING**